



Name: Jaijishan Saharan

Course: B.Tech CSE-SY (Core)

Division: D

Roll No: 26

DSA PRACTICAL ASSIGNMENT 2

Aim: write a C++ program to sort an array of N elements using bubble & selection sort.

Objectives:

- To understand bubble sort and its implementation.
- To understand selection sort and its implementation.

Conclusion: ¶

Thus we have sorted an array of N elements using bubble and selection sort using an array as a data structure.

Questions

Q1. What are bubble sort and selection sort. Write an algorithm for bubble and selection sort?

Sol. ① • Bubble sort is a simple comparison-based sorting algorithm.

- It repeatedly compares adjacent elements in the array.
- If they are in wrong order, it swaps them.
- After each pass, the largest element "bubbles up" to the end.

Algorithm for bubble sort

Bubblesort (A, n) // A is the array, n is the number of elements.

- 1) Repeat for $i = 0$ to $n-2$
- 2) Repeat for $j = 0$ to $n-1-i$
- 3) If $A[j] > A[j+1]$, then
- 4) Swap $A[j]$ and $A[j+1]$
- 5) End.

② • Selection Sort is another simple comparison based sorting algorithm.

- It repeatedly finds the smallest element in the unsorted part of the array.
- Then it swaps it with the first unsorted element.
- After each pass the sorted part while unsorted part shrinks.



Algorithm for selection sort:

1. Start: Initialize an array of n elements.
2. Outer Loop: For each element from 0 to $n-1$:
 - a) Assume the first unsorted element is the minimum.
3. Inner Loop: For each element from $i+1$ to $n-1$:
 - a) Find the minimum element in the unsorted subarray.
 - b) If a smaller element is found, update the minimum.
4. Swap: After finding the minimum element in the unsorted subarray, swap it with the first unsorted element.
5. Repeat: Continue this process until the entire array is sorted.
6. End: The array is now sorted.

Q2 Sort the element: $[64, 25, 12, 22, 11]$ using Bubble Sort and selection sort. Give the output in each pass and finally the sorted array.

Sol Bubble sort.

Array = $[64, 25, 12, 22, 11]$

64	25	12	22	11
25	64	12	22	11
25	12	64	22	11
25	12	22	64	11
25	12	22	11	64

} Pass 1.



25	12	22	11	64	} Pass 2
12	25	22	11	64	
12	22	25	11	64	
12	22	11	25	64	
12	22	11	25	64	

12	22	11	25	64	} Pass 3
12	22	11	25	64	
12	11	22	25	64	
12	11	22	25	64	
12	11	22	25	64	

12	11	22	25	64	} Pass 4
11	12	22	25	64	

→ Sorted Array: [11, 12, 22, 25, 64]

Selection Sort

Array = [64, 25, 12, 22, 11]

- Pass 1:

Find minimum (11), swap with first element (64)

→ [11, 25, 12, 22, 64]

- Pass 2:

Find minimum (12) from remaining → swap with 25

→ [11, 12, 25, 22, 64]



- Pass 3:

Find minimum (22) from remaining $\rightarrow$ swap with 25
 $\rightarrow [11, 12, 22, 25, 64]$

- Pass 4:

Find the minimum (25) $\rightarrow$ already in correct place.
 $\rightarrow [11, 12, 22, 25, 64]$

$\rightarrow$ Sorted Array $\rightarrow [11, 12, 22, 25, 64]$

Q3 what is the time complexity of Bubble sort and Selection sort.

1) Bubble Sort:

- Repeatedly compares adjacent elements and swaps them if they are in wrong order.
- Time complexity:
 - Best case $\rightarrow O(n)$ $\rightarrow$ Array is already sorted
 $\rightarrow$ only one pass, with no swaps.
 - Average case $\rightarrow O(n^2)$ $\rightarrow$ Typical unsorted array requires multiple passes.
 - Worst case $\rightarrow O(n^2)$ $\rightarrow$ Array is reverse sorted
 $\rightarrow$ maximum comparisons & swaps.

2) Selection Sort:

- Finds the minimum element in the unsorted portion and swaps it with the first unsorted element.

Time complexity:

- Best case $\rightarrow O(n^2)$ $\rightarrow$ Even if the array is sorted it still looks for the minimum each time.
- Average case $\rightarrow O(n^2)$ $\rightarrow$ standard unsorted array
- Worst case $\rightarrow O(n^2)$ $\rightarrow$ Reverse sorted $\rightarrow$ same as average.

Q4. What are the main advantages and disadvantages of Bubble Sort and selection sort?

Sol. Bubble sort:

Advantages:

1. Simple to understand and implement.
2. In place sorting - does not require extra memory.
3. Stable - maintains the relative order of equal elements.
4. Optimizable for best case - can detect already sorted arrays and finish early, giving $O(n)$ time.

Disadvantages:

1. Inefficient for large arrays - Average and worst case time is $O(n^2)$.
2. Many swaps - swapping elements frequently is constant.
3. Not suitable for large datasets - performance is poor compared to other sorting algorithms like quicksort or mergesort.



2) Selection sort

Advantages

1. Simple to understand and implement.
2. In-place sorting-only requires a constant amount of extra memory.
3. Fewer swaps than bubble sort-swaps only once per pass.

Disadvantages

1. Inefficient for large arrays. Time complexity $O(n^2)$ in all cases.
2. Unstable - Equal elements may change order after sorting.
3. Does not benefit from partially sorted arrays.
- Always perform same number of comparisons.

Q5. Can Bubble Sort be used for sorting real world applications?

Ans. It is generally not suitable for real-world applications due to its poor performance on large datasets. With a complexity of $O(n^2)$, Bubble Sort becomes inefficient as the size of the data grows, making it impractical for most modern computing needs. However, it may be used in educational settings to teach basic algorithm concepts or in scenarios involving very small or nearly sorted datasets where performance is not critical.



Q6. How can you optimize Bubble sort?

Sol. Use of a swap flag

- In the normal bubble sort, the algorithm always make $n-1$ passes even if the array becomes sorted before that.
 - By introducing a flag variable, we can check if any swap has occurred in the current pass.
 - If no swap is made, it means the array is already sorted, and we can stop early.
 - This reduces unnecessary passes and improves efficiency for nearly sorted arrays.
- Best case performance.

- With the above optimization, bubble sort's best case time complexity becomes $O(n)$.
- Worst-case complexity remains $O(n^2)$, but performance improves for many practical cases.

Q7. How can you optimize selection sort?

Sol. Selection sort can be optimized by:

- Selecting both minimum and maximum elements in each pass (placing the minimum at the beginning and maximum at the end), which reduces the total no. of passes.
- Avoiding unnecessary swaps by only swapping when the current element is not already in the correct position.
- This will reduce the number of operations.



Q8. How is the Selection sort different from other sorting algorithms like Bubble sort and insertion sort?

801. <u>Selection sort</u>	<u>Bubble sort</u>	<u>Insertion sort</u>
finds the smaller (or largest) element and places it in the correct position	Repeatedly swaps adjacent elements to move largest to its correct position.	Inserts each new element into its correct position in the already sorted part.
Performs fewer swaps (at most $n-1$)	Performs many swaps (almost every comparison may swap)	Uses element shifts instead of many swaps.
Time complexity is always $O(n^2)$	Best case $O(n)$ if already sorted otherwise $O(n^2)$	Best case $O(n)$ for nearly sorted data, otherwise $O(n^2)$
not stable	stable	stable

Conclusion :

24/10/19